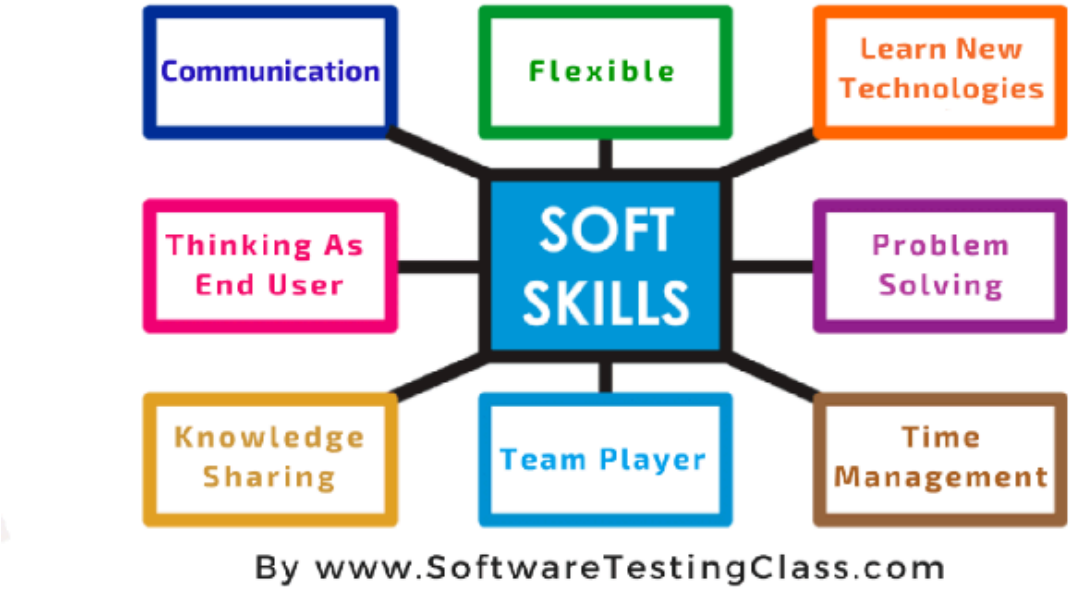
TBefore we begin: What are soft skills?

Soft skills are a combination of communication, emotional, social, and interpersonal skills that allow us to work well with others. They are often overlooked; however, in my opinion, these skills are quite important in the software development world, and every programmer should understand and strive to develop them for their career. Certainly, good coding is important, but I tend to place both hard skills and soft skills on equal importance for every programmer. In my view, a great programmer also needs to be a great teammate. The two go hand in hand.



Develop empathy

Empathy is the ability to understand other people's feelings. For a moment, imagine yourself in someone else's shoes and try to think about what that person likes. In my opinion, this is the foundation of all interaction. As programmers who collaborate with many people, some of whom you may like or dislike, our empathy is challenged many times each day, in many situations, such as:

- when pairing and reviewing
- when collecting specifications
- when debugging for others
- when leading other programmers

The importance of language

Language and communication have a huge impact on our daily lives: too often, I find that people feel inferior when they are belittled or rubbed off due to cultural and language-related skills rather than a lack of empathy; especially for some people who are not 100% fluent in a particular language, they are often more easily misunderstood than native speakers.
Due to the diversity of the teams I work with, situations like this happen daily; sometimes it's easy to get discouraged. When someone speaks a different language, you're faced with two problems: word choice and cultural aspects. Of course, word choice can be influenced by cultural factors: for example, in English, it's common to use apologies, thank you, and please in sentences. However, this is rare in Vietnamese. Of course, I hope others will know this and use similar words when necessary.

Listen carefully, then speak.

This is perhaps universally true common sense, but it needs to be stated. As programmers, after all, we strive to be efficient in every aspect of our lives.
Interrupting your colleagues is one of the most common behaviors I see in meetings. It's a surefire way to frustrate your coworker, in case you're wondering.
Listening first and carefully is incredibly important, and it's not just a matter of respect, even if you don't agree with every word. You might think it's a waste of time, but it's true.

Teaching, but also learning at the same time.

It's very easy to lower your voice when you're trying to make your point, explain something, or while advising someone else. Communication is incredibly difficult, so you should expect this to happen even if you absolutely don't intend it to. There are many ways we can try to avoid lowering our voice, such as doing so during discussions.

- Of course, choosing your words carefully is important, and my advice is to be as neutral as possible: for example, instead of saying it's a bad approach, I would say it's not an optimal approach, or that it could be better.
- Stop commanding, instead advise: instead of saying, "You should do it that way," you can say, "My advice is to do it that way."
- Stop just stating your point of view; ask questions frequently and check your interlocutor's perspective on what you're discussing.
- Stop assuming you know everything about this topic.

Accept criticism and be open to change.

We are a creative industry.
For us, our code is art, like a painting to an artist or a song to a musician.
Becoming a programmer means being judged every day, in one way or another. When we tend to identify personally with our output, we make any judgments about our code, our ideas, and our opinions extremely central. One of the hardest parts of our job is accepting criticism from others, whether in interviews, meetings, PR reviews, etc.

We spend years honing our skills in a particular technology, and as time goes on, we become accustomed to our way of working or the way our team does. As a result, we become increasingly resistant to change, especially when it comes from a third party.
Remember that rookie who wanted to start restructuring the entire codebase? Or someone who wanted to introduce Go and replace your Java? I'm sure we've all, at some point, experienced situations like that: I don't know about you, but for me, they were a personal attack.
Understandably, developers hate change.
If you're not paying attention, this will happen in two ways:

- The new recruit liked the architecture at his previous company and lacked the patience to rewrite everything the way he used it, because he was 100% certain that it was better.
- Instead, the team was accustomed to how they wrote their codebase (or the way they inherited it) and had no intention of letting newcomers change things.

Of course, this is a general (though very common) scenario, and either side could be right. Perhaps the current architecture is genuinely bad and needs restructuring, and perhaps the guy is just being overly concerned.
The problem is that if you immediately feel attacked when someone else brings up the fact that you want to change something, then the problem is with you, not with the idea itself. You've been irrational in defending yourself against a new approach before fully understanding it.
I think this happens to me all the time. It's so similar to me and so many other people. How can I be sure that I'm not fighting this idea because of myself?

- I listened to it carefully.
- I asked my colleague to give me some time to do some research and think about it.
- I'm returning to my honest thoughts.

General advice

- If you think a colleague has done a good job, offer them genuine support and praise.
- Provide credit to others when it is due.
- The more transparent, the better: talk to your colleagues about clarifications, changes, and feedback. For example, secretly making commitments when someone is out of the office isn't a good way to refactor someone else's code. Talk and think about it together openly.
- Sometimes conflict will arise if you've done nothing wrong and followed all possible advice; remember, this is perfectly normal, we can control things. Companies and people are complex, and this is simply a simplification of what actually happens on a daily basis.